

VISVESVARAYA TECHNOLOGICAL UNIVERSITY



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

PRIYANKA CHOURASIA(1BF24CS233)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF
ENGINEERING (Autonomous
Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S.CollegeofEngineering,
Bull Temple Road,Bangalore 560019
(Affiliated To Visvesvaraya Technological University,
Belgaum) Department of ComputerScience and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHM**” carried out by **Priyanka Chourasia (1BF24CS233)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab (**23CS4PCADA**) work prescribed for the said degree.

Shwetha K S
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	4-6
2	Implement Johnson Trotter algorithm to generate permutations.	7-9
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	10-12
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	13-14
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	15-17
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	18-20
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	21-23
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	24-28
9	Implement Fractional Knapsack using Greedy technique.	29-31
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	32-34
11	Implement "N-Queens Problem" using Backtracking.	35-36

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem. Conduct practical experiments by applying appropriate algorithm design
CO4	technique to solve problems.

Lab program1:

Write program to obtain the Topological ordering of vertices in a given digraph.

Code:

```
#include<stdio.h>

#define MAX 10

int main() {
    int n, a[MAX][MAX];
    int indegree[MAX] = {0};
    int visited[MAX] = {0};
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    //Input adjacency matrix and calculate indegree
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            scanf("%d", &a[i][j]);
            if(a[i][j] == 1) {
                indegree[j]++;
            }
        }
    }
    printf("Topological Order: ");
    for(int count = 0; count < n; count++) {
        for(int i = 0; i < n; i++)
            if(indegree[i] == 0 && visited[i] == 0) {
                printf("%d ", i);
```

```
visited[i]=1;
```

```
// Reduce indegree of adjacent vertices
```

```
    for(int j = 0; j < n; j++) {  
        if(a[i][j] == 1) {  
            indegree[j]--;  
        }  
    }  
}  
}  
}  
}  
return 0;  
}
```

Output>>>



```
PS C:\Users\91876\Desktop\ada> cd "c:\Users\91876\Desktop\ada\" ; if ($?) {  
    .\topological_sorting } ; if ($?) { .\topological_sorting }  
Enter number of vertices: 6  
Enter adjacency matrix:  
0 0 0 0 0 0  
0 0 0 0 0 0  
0 0 0 1 0 0  
0 1 0 0 0 0  
1 1 0 0 0 0  
1 0 1 0 0 0  
Topological Order:  
4 5 0 2 3 1  
PS C:\Users\91876\Desktop\ada>
```

LeetCode Program:

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int*
prerequisitesColSize) {
    int** graph = (int**)malloc(numCourses * sizeof(int*));
    int* graphColSize = (int*)calloc(numCourses, sizeof(int));
    for (int i = 0; i < numCourses; i++) {
        graph[i] = (int*)malloc(numCourses * sizeof(int));
    }
    int* indegree = (int*)calloc(numCourses, sizeof(int));
    for (int i = 0; i < prerequisitesSize; i++) {
        int a = prerequisites[i][0];
        int b = prerequisites[i][1];
        graph[b][graphColSize[b]++] = a; indegree[a]++;
    }
    int* queue = (int*)malloc(numCourses * sizeof(int));
    int front = 0, rear = 0;
    for (int i = 0; i < numCourses; i++)
        { if (indegree[i] == 0) {
            queue[rear++] = i;
        } }
    int count = 0;
    while (front < rear) {
        int course = queue[front++];
        count++;
        for (int i = 0; i < graphColSize[course]; i++) {
            int neighbor = graph[course][i];
            indegree[neighbor]--;
```

```

if (indegree[neighbor] == 0)
{
    queue[rear++] =
neighbor;
}}
return count ==
numCourses;
}

```



Lab program 2:

Implement Johnson Trotter algorithm to generate permutations.

Code:

```
#include<stdio.h>

#define RIGHT_TO_LEFT 0
#define LEFT_TO_RIGHT 1

//searchpositionofmobileelement
intsearchArr(inta[],intn,int mobile) {

    for(int i=0;i<n;i++) {

        if(a[i]==mobile)
            return i;
    }
}

// find mobile element intgetMobile(inta[],intdir[],
int n)
{
    intmobile_prev=0,mobile = 0;

    for(int i=0;i<n;i++)
    {
        // RIGHT TO LEFT
        if(dir[a[i]-1]==RIGHT_TO_LEFT && i!=0)
        {
            if(a[i]>a[i-1]&&a[i] > mobile_prev)
            {
                mobile = a[i]; mobile_prev=mobile;
            }
        }

        // LEFT TO RIGHT
        if(dir[a[i]-1]==LEFT_TO_RIGHT && i!=n-1)
        {
            if(a[i]>a[i+1]&&a[i] > mobile_prev)
            {
                mobile = a[i]; mobile_prev=mobile;
            }
        }
    }

    if(mobile==0&&mobile_prev==0) return 0;
    else
```

```

        return mobile;
    }

// print one permutation void printOnePerm(int a[], int
dir[], int n)
{
    int mobile = getMobile(a, dir, n);

    int pos = searchArr(a, n, mobile);

    // swap
    if (dir[a[pos]-1] == RIGHT_TO_LEFT)
    {
        int temp = a[pos];
        a[pos] = a[pos-1];
        a[pos-1] = temp;
    }

    elseif (dir[a[pos]-1] == LEFT_TO_RIGHT)
    {
        int temp = a[pos];
        a[pos] = a[pos+1];
        a[pos+1] = temp;
    }

    // reversed directions
    for (int i = 0; i < n; i++)
    {
        if (a[i] > mobile)
        {
            if (dir[a[i]-1] == LEFT_TO_RIGHT)
                dir[a[i]-1] = RIGHT_TO_LEFT;

            elseif (dir[a[i]-1] == RIGHT_TO_LEFT)
                dir[a[i]-1] = LEFT_TO_RIGHT;
        }
    }

    // print permutation
    for (int i = 0; i < n; i++)
        printf("%d", a[i]);

    printf("\n");
}

// factorial int
fact(int n)
{
    int res = 1; for (int i = 1; i <= n; i++)
        res = res * i;
    return res;
}

```

```

        res = res * i; return
    res;
}

int main()
{
    int n; printf("Enter n:
    ");
    scanf("%d",&n);

    int a[n], dir[n];
    // initialize
    for(int i=0;i<n;i++)
    {
        a[i] = i + 1;
        dir[i] = RIGHT_TO_LEFT;
    }

    // print first permutation for(int
    i=0;i<n;i++) printf("%d ",a[i]);

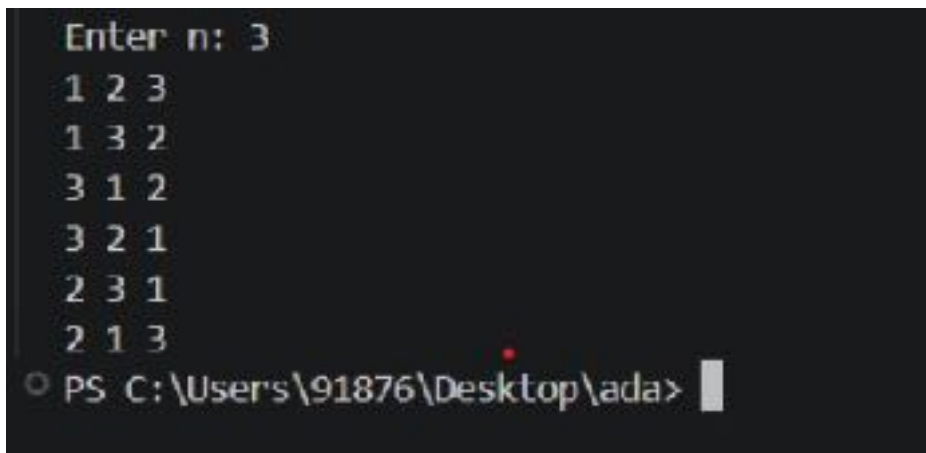
    printf("\n");

    // generate permutations for(int
    i=1;i<fact(n);i++)
        printOnePerm(a, dir, n);

    return 0;
}

```

Output>>>



```

Enter n: 3
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3
PS C:\Users\91876\Desktop\ada>

```

Lab program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

Code: #include<stdio.h>

#include<time.h>

```
void merge(int a[], int low, int mid, int high)
```

```
{
    int i = low, j = mid + 1, k = low; int
    temp[100];
```

```
    while(i <= mid && j <= high)
    {
        if(a[i] < a[j]) temp[k++] = a[i+
            +];
        else
            temp[k++] = a[j++];
    }
```

```
    while(i <= mid) temp[k++] = a[i++];
```

```
    while(j <= high) temp[k++] = a[j++];
```

```
    for(i = low; i <= high; i++) a[i] =
        temp[i];
}
```

```
void mergesort(int a[], int low, int high)
```

```
{
    if(low < high)
    {
        int mid = (low + high) / 2;

        mergesort(a, low, mid); mergesort(a, mid
            + 1, high); merge(a, low, mid, high);
    }
}
```

```
int main()
```

```
{
```

```

int a[100], n, i; clock_t
start, end;

printf("Enter number of elements: "); scanf("%d", &n);

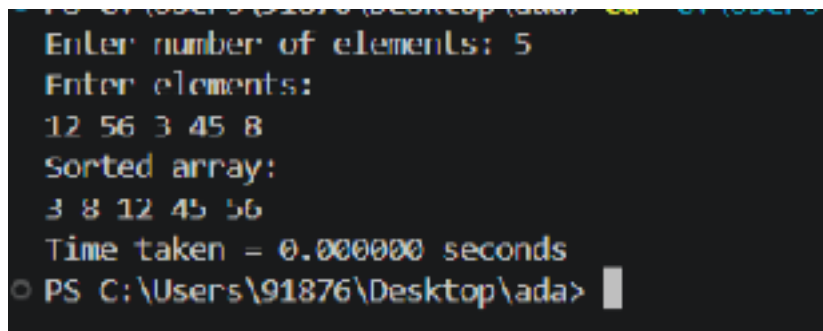
printf("Enter elements:\n"); for(i =
0; i < n; i++)
    scanf("%d", &a[i]); start
= clock(); mergesort(a, 0, n -
1); end = clock();

printf("Sorted array:\n");
for(i = 0; i < n; i++)
printf("%d ", a[i]);

printf("\nTime taken = %lf seconds", (double)(end -
start) / CLOCKS_PER_SEC)
return 0;
}

```

Output>>>

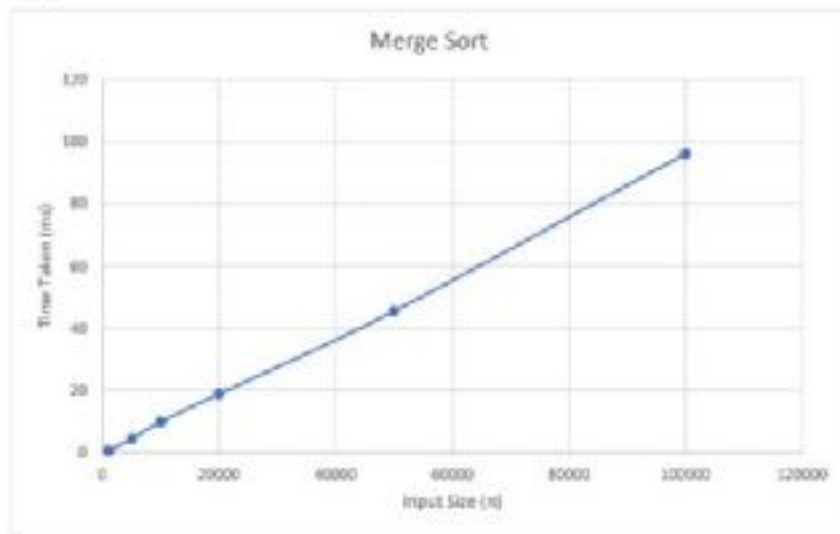


```

Enter number of elements: 5
Enter elements:
12 56 3 45 8
Sorted array:
3 8 12 45 56
Time taken = 0.000000 seconds
PS C:\Users\91876\Desktop\ada>

```

Graph:



LEETCODE PROGRAM

intreverseBits(int n)

```
{ unsigned int result = 0;
```

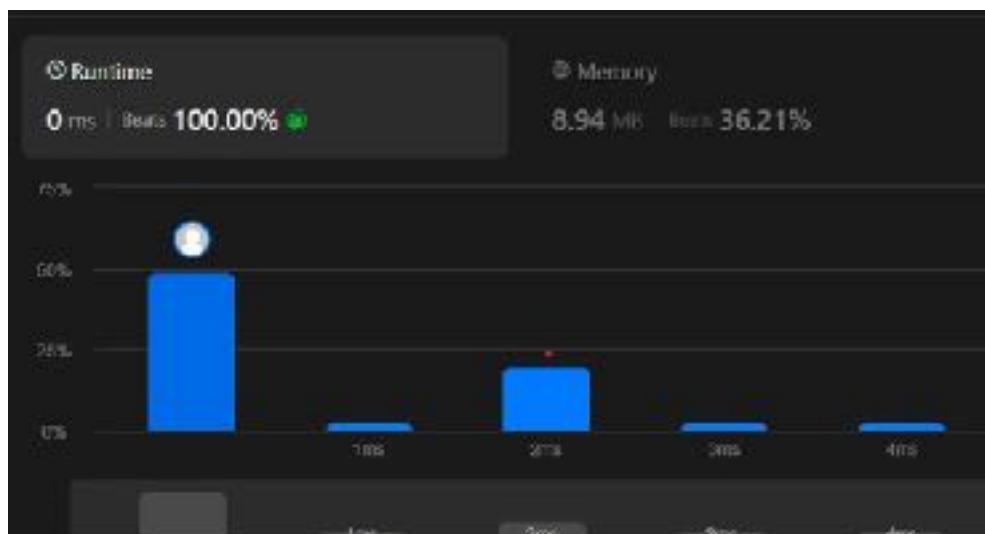
```
unsigned int num = (unsigned int)n;
```

```
for(int i = 0; i < 32; i++) {
```

```
    result = (result << 1) | (num & 1);
```

```
    num >>= 1;
```

```
}
```



Lab program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

Code: #include<stdio.h>
#include<time.h>

```
int partition(int a[], int low, int high)
{
    int pivot = a[low]; int
    i = low + 1; int j =
    high;
    int temp;

    while(i <= j)
    {
        while(a[i] <= pivot && i <= high) i++;

        while(a[j] > pivot) j--;

        if(i < j)
        {
            temp = a[i]; a[i]
            = a[j]; a[j] =
            temp;
        }
    }

    temp = a[low];
    a[low] = a[j]; a[j]
    = temp;

    return j;
}

void quicksort(int a[], int low, int high)
{
    if(low < high)
    {
        int p = partition(a, low, high);

        quicksort(a, low, p - 1); quicksort(a, p +
        1, high);
    }
}
```

```

    }
}

int main()
{
    int a[100], n, i; clock_t
    start, end;
    printf("Enter number of elements: "); scanf("%d", &n);

    printf("Enter elements:\n"); for(i =
    0; i < n; i++)
        scanf("%d", &a[i]);

    start = clock();

    quicksort(a, 0, n - 1); end

    = clock();

    printf("Sorted array:\n");
    for(i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\nTime taken = %lf seconds",
        (double)(end - start) / CLOCKS_PER_SEC);
    return 0;
}

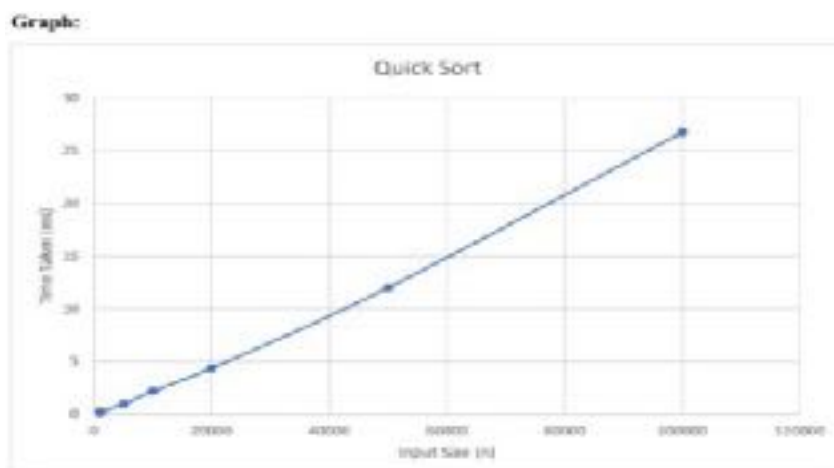
```

Output>>>

```

Enter number of elements: 5
Enter elements:
12 34 3 4 67
Sorted array:
3 4 12 34 67
Time taken = 0.000000 seconds
PS C:\Users\91876\Desktop\ada>

```



Lab program 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include <stdio.h> voidminHeapify(intarr[], int n,
int i)
{
    int smallest=i;
    intleft=2*i+1;
    int right = 2 * i + 2; if(left<n&&arr[left]<
arr[smallest]) smallest = left;
    if(right<n&&arr[right] < arr[smallest]) smallest =
    right;
    if (smallest != i)
    {
        int temp = arr[i]; arr[i]=
        arr[smallest]; arr[smallest]
        = temp;
        minHeapify(arr,n,smallest);
    }
}

voidbuildMinHeap(intarr[], int n)
{
    for(inti=(n/2)-1;i>= 0; i--)
    {
        minHeapify(arr, n, i);
    }
}
```

```

void heapSortDescending(int arr[], int n) { buildMinHeap(arr, n);

    for (int i = n - 1; i > 0; i--)

    {

        int temp = arr[0]; arr[0]
        = arr[i]; arr[i] = temp;
        minHeapify(arr, i, 0);
    }

}

void printArray(int arr[], int n) { for (int i
= 0; i < n; i++){ printf("%d ", arr[i]);
}

printf("\n");
}

int main()

{

    int arr[] = {40, 10, 30, 50, 60, 15, 5}; int n = sizeof(arr) / sizeof(arr[0]); printf("Input Array:\n"); printArray(arr, n);
    buildMinHeap(arr, n); printf("\nMin
    Heap:\n"); printArray(arr, n);
    heapSortDescending(arr, n);
    printf("\nDescending Order:\n");
    printArray(arr, n);
    return 0;}

```

Output>>>

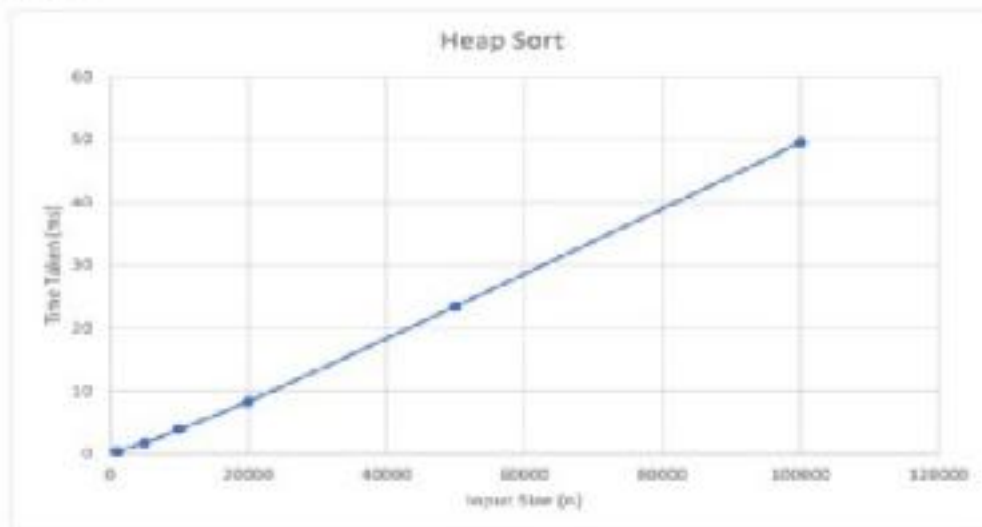
```
Input Array:
48 10 30 50 60 15 5

Min Heap:
5 10 15 50 60 40 30

Descending Order:
60 50 40 30 15 10 5

Process returned 0 (0x0)   execution time : 0.007 s
Press any key to continue.
```

Graph:



Lab program 6:

Implement 0/1 Knapsack problem using dynamic programming.

Code:

```
#include<stdio.h> int
max(int a, int b){
    if(a>b){
        return a;
    }
    else
        return b;
}
int main(){ int
    w,n;
    printf("Enter no of items:");
    scanf("%d",&n);
    int wt[n], p[n]; for(int
    i=0;i<n;i++){
        scanf("%d%d",&wt[i],&p[i]);
    }
    printf("Enter the capacity of sack:"); scanf("%d",&w);
    int k[n+1][w+1]; for(int
    i=0;i<=n;i++){
        for(int j=0;j<=w;j++){ if(i==0
            || j==0){
                k[i][j]=0;
```

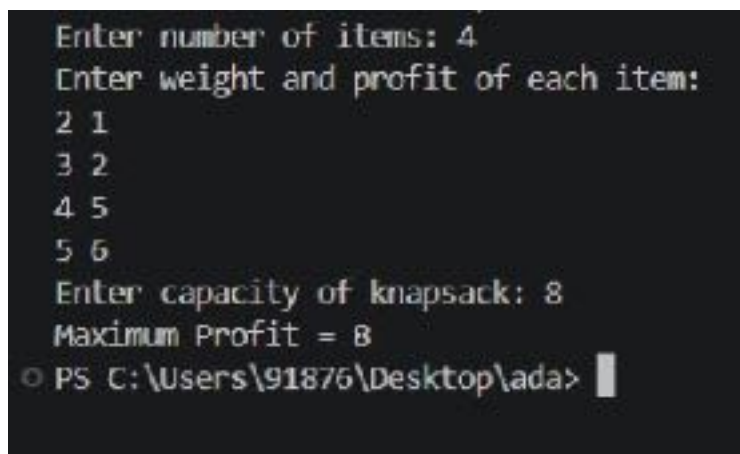
```

    }
    else if(wt[i-1]<=j){
        k[i][j]=max(p[i-1]+k[i-1][j-wt[i-1]],k[i-1][j]);
    }
    else{
        k[i][j]=k[i-1][j];
    }
}
}

printf("maximum profit:%d" ,k[n][w]);
}

```

Output>>>



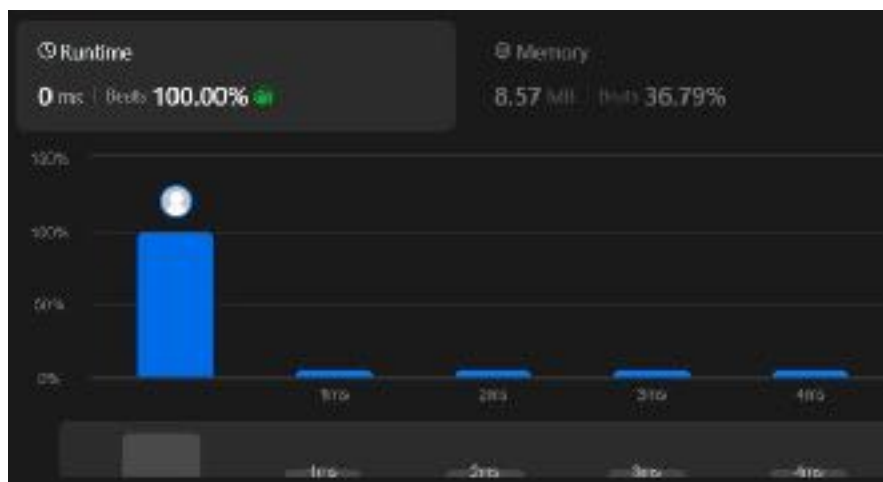
```

Enter number of items: 4
Enter weight and profit of each item:
2 1
3 2
4 5
5 6
Enter capacity of knapsack: 8
Maximum Profit = 8
PS C:\Users\91876\Desktop\ada>

```

LEETCODE PROGRAM:

```
int climbStairs(int n) {  
  
    if(n==1)  
  
        return 1;  
  
    int dp[n+1];  
  
    dp[1]=1;  
  
    dp[2]=2;  
  
    for(int i=3;i<=n;i++){  
  
        dp[i]=dp[i-1]+dp[i-2];  
  
    }  
  
    return dp[n];}
```



LAB PROGRAM-7

Implement All Pair Shortest paths problem using Floyd's algorithm. Code:

```
#include <stdio.h> int
main()
{
int n;
int a[10][10];
printf("Enter number of vertices: ");
scanf("%d",&n);
printf("Enter cost matrix:\n"); for(int
i=0;i<n;i++)
for(int j=0;j<n;j++)
scanf("%d",&a[i][j]); for(int
k=0;k<n;k++)
{
for(int i=0;i<n;i++)
{
for(int j=0;j<n;j++)
{
if(a[i][j] > a[i][k]+a[k][j])
a[i][j]=a[i][k]+a[k][j];
}
}
}
}

printf("Shortest path matrix:\n");
```

```

}

for(int i=0;i<n;i++)

{

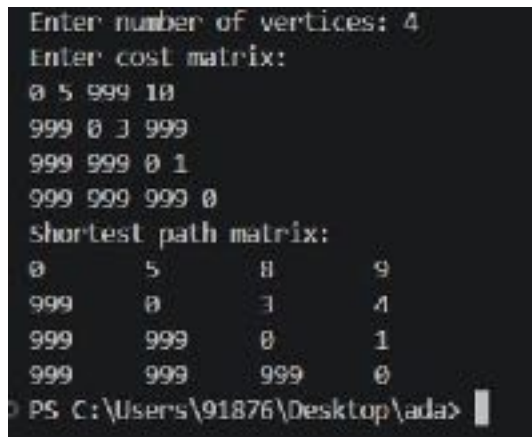
    for(int j=0;j<n;j++) printf("%d

    ",a[i][j]); printf("\n");

}

```

Output>>>



```

Enter number of vertices: 4
Enter cost matrix:
0 5 999 10
999 0 3 999
999 999 0 1
999 999 999 0
Shortest path matrix:
0      5      8      9
999     0      3      4
999     999     0      1
999     999     999     0
PS C:\Users\91876\Desktop\ada>

```

LEETCODEPROGRAM:

```

intcountWays(int nums[], int n, int index, int sum, int target) { if(index == n) {

return (sum == target) ? 1 : 0;

}

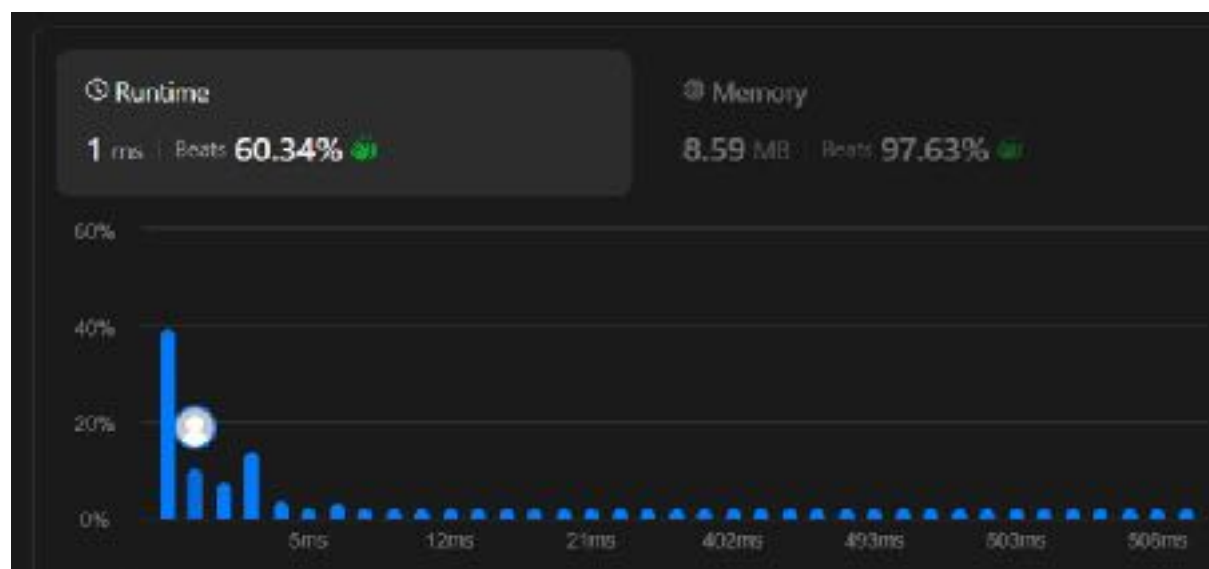
intadd = countWays(nums, n, index + 1, sum + nums[index], target); int subtract = countWays(nums, n, index
+ 1, sum - nums[index], target); return add + subtract;

}

intfindTargetSumWays(int nums[], int n, int target) { return

countWays(nums, n, 0, 0, target);

```



LAB PROGRAM-8

(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

Code:

```
#include<stdio.h>

#define INF 999 int

main()
{
    int n, i, j, min, u, v; int
    visited[10]= {0}; int
    cost[10][10];

    int mincost = 0, edges = 0; printf("Enter
    number of vertices: "); scanf("%d", &n);
    printf("Enter cost matrix:\n"); for(i =
    0; i < n; i++)
    {
        for(j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);

            if(cost[i][j] == 0)
                cost[i][j] = INF;
        }
    }
    visited[0] = 1;
    printf("Edges in Minimum Spanning Tree:\n");
    while(edges < n - 1)
    {
```

```

min = INF;

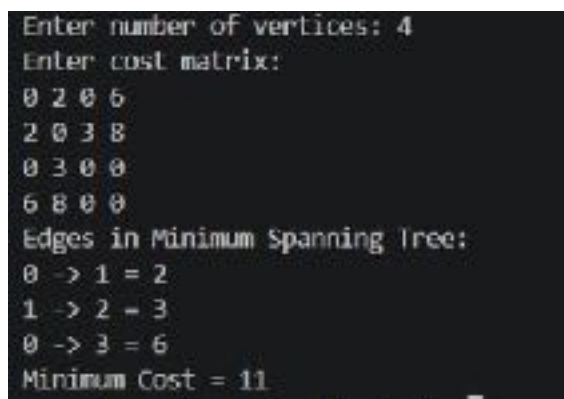
for(i = 0; i < n; i++){ if(visited[i]){
    for(j = 0; j < n; j++){ if(!visited[j] &&
        cost[i][j] < m){
            min = cost[i][j]; u = i;
            v = j;
        }
    }
}

printf("%d -> %d = %d\n", u, v, min); mincost
+= min;
visited[v] = 1; edges+
+;
}

printf("Minimum Cost = %d", mincost); return 0;
}

```

Output>>>



```

Enter number of vertices: 4
Enter cost matrix:
0 2 0 6
2 0 3 8
0 3 0 0
6 8 0 0
Edges in Minimum Spanning Tree:
0 -> 1 = 2
1 -> 2 = 3
0 -> 3 = 6
Minimum Cost = 11

```

(b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

Code:

```
#include<stdio.h>

#define INF 999 int

parent[10];

int find(int i)
{
    while(parent[i] != i)
        i = parent[i];
    return i;
}

int uni(int i, int j)
{
    if(i != j)
    {
        parent[j] = i; return 1;
    }
    return 0;
}

int main()
{
    int n, i, j, a, b, u, v; int
    min, mincost = 0; int
    cost[10][10];
    int edges = 0;
    printf("Enter number of vertices: "); scanf("%d", &n);
```

```

printf("Enter cost matrix:\n"); for(i = 0;
i < n; i++)
{
    for(j=0;j<n;j++)
    {
        scanf("%d",&cost[i][j]);

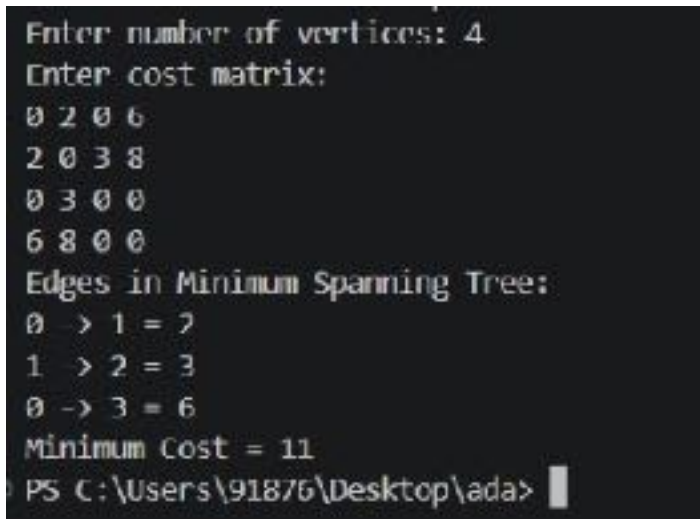
        if(cost[i][j]==0)
            cost[i][j]=INF;
    }
}
printf("Edges in Minimum Spanning Tree:\n") while(edges
< n- 1)
{
    min = INF
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            if(cost[i][j]<min)
            {
                min=cost[i][j]; a=u=i;
                b=v=j;

            }
        }
    }
}

```

```
u = find(u); v = find(v); if(uni(u, v))
{
    printf("%d -> %d = %d\n", a, b, min); mincost +=
    min;
    edges++;
}
cost[a][b] = cost[b][a] = INF;
}
printf("Minimum Cost = %d", mincost); return 0;
}
```

Output>>>



```
Enter number of vertices: 4
Enter cost matrix:
0 2 0 6
2 0 3 8
0 3 0 0
6 8 0 0
Edges in Minimum Spanning Tree:
0 -> 1 = 2
1 -> 2 = 3
0 -> 3 = 6
Minimum Cost = 11
PS C:\Users\91876\Desktop\ada>
```

LAB PROGRAM-9:

Implement Fractional Knapsack using Greedy technique.

```
#include<stdio.h> struct
```

```
Item
```

```
{
```

```
    int profit, weight; float
```

```
    ratio;
```

```
};
```

```
int main()
```

```
{
```

```
    int n, i, j;
```

```
    float capacity, totalProfit = 0; printf("Enter
```

```
    number of items: "); scanf("%d", &n)
```

```
    struct Item item[n];
```

```
    printf("Enter profit and weight of each item:\n"); for(i = 0; i < n;
```

```
    i++)
```

```
    {
```

```
        scanf("%d%d", &item[i].profit, &item[i].weight); item[i].ratio =
```

```
        (float)item[i].profit / item[i].weight;
```

```
    }
```

```
    printf("Enter capacity of knapsack: "); scanf("%f",
```

```
    &capacity);
```

```
    // Sort by profit/weight ratio for(i
```

```
    = 0; i < n - 1; i++)
```

```
    {
```

```
        for(j = i + 1; j < n; j++)
```

```

    {
        if(item[i].ratio < item[j].ratio)
        {
            struct Item temp = item[i]; item[i]
            = item[j];
            item[j] = temp;
        }
    }
}
for(i=0; i < n; i++)
{
    if(capacity >= item[i].weight)
    {
        capacity -= item[i].weight; totalProfit
        += item[i].profit;
    }
    else
    {
        totalProfit += item[i].ratio * capacity; break;
    }
}
printf("Maximum Profit = %.2f", totalProfit); return 0;
}

```

Output>>>

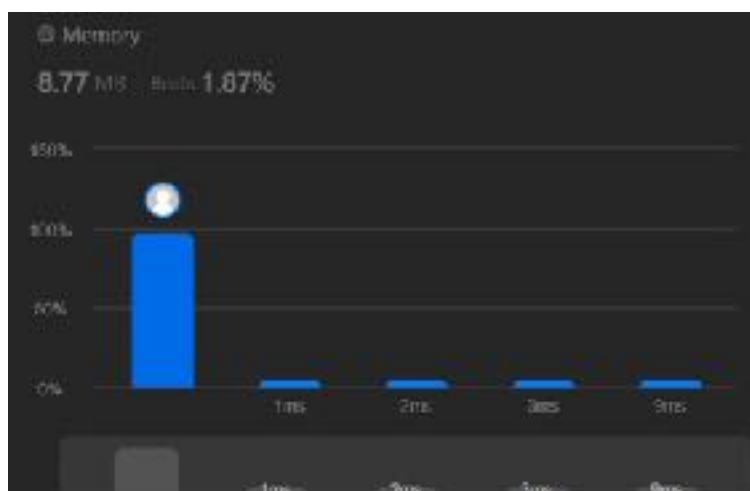
```
Enter number of items: 3
Enter profit and weight of each item:
60 10
100 20
120 30
Enter capacity of knapsack: 50
Maximum Profit = 240.00
PS C:\Users\91876\Desktop\ada>
```

LEET CODE:

```
int balancedStringSplit(char* s) {

    int balance = 0; int
    count = 0;

    for (int i = 0; s[i] != '\0'; i++) { if (s[i] ==
    'R')
    balance++;
    else balance--;
    if (balance == 0) count++;
    }
    return count;
}
```



LAB PROGRAM -10

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

Code:

```
#include<stdio.h>

#define INF 999 int

main()
{
    int n, i, j, u, v, min; intvisited[10]= {0};
    intcost[10][10],dist[10];
    printf("Enternumberofvertices: ");
    scanf("%d", &n); printf("Entercostmatrix:
    \n");
    for(i=0; i<n; i++)
    {
        for(j=0; j<n; j++)
        {
            scanf("%d",&cost[i][j]);

            if(cost[i][j] == 0)
                cost[i][j] = INF;
        }
    }
    int source; printf("Entersourcevertex: ");
    scanf("%d", &source);
```

```

// Initialize distances for(i =
0; i < n; i++)
{
    dist[i]=cost[source][i];
}
visited[source] = 1;
dist[source] = 0; for(i = 1;
i < n; i++)
{
    min = INF;
    for(j=0;j<n;j++)
    {
        if(!visited[j]&&dist[j] < min)
        {
            min = dist[j];
            u=j;
        }
    }
    visited[u] = 1; for(v=0;v<n;v++)
    {
        if(!visited[v]&&dist[u] + cost[u][v] < dist[v])
        {
            dist[v]=dist[u] + cost[u][v];
        }
    }
}

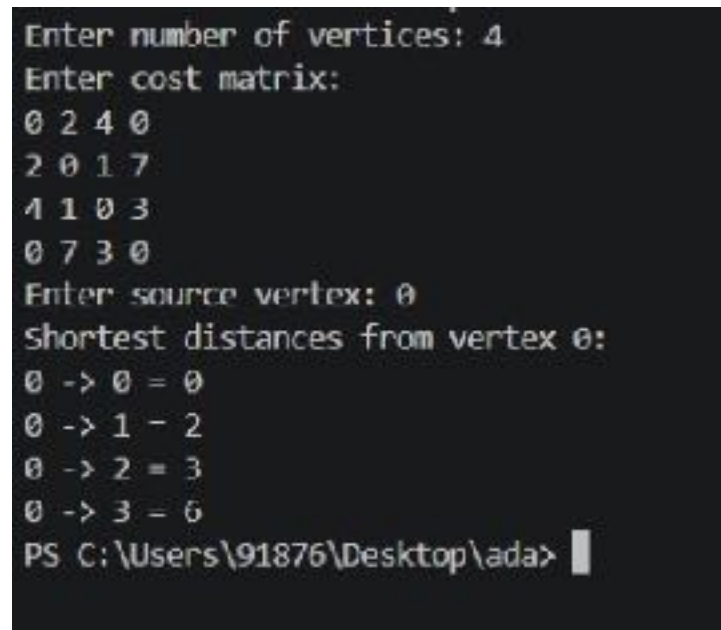
```

```

printf("Shortest distances from vertex %d:\n", source); for(i = 0; i < n;
    i++)
{
    printf("%d -> %d = %d\n", source, i, dist[i]);
}
return 0;
}

```

Output>>>



```

Enter number of vertices: 4
Enter cost matrix:
0 2 4 0
2 0 1 7
4 1 0 3
0 7 3 0
Enter source vertex: 0
Shortest distances from vertex 0:
0 -> 0 = 0
0 -> 1 = 2
0 -> 2 = 3
0 -> 3 = 6
PS C:\Users\91876\Desktop\ada>

```

LAB PROGRAM-11

Implement “N-Queens Problem” using Backtracking.

Code:

```
#include<stdio.h> int
x[10], count = 0; int
place(int k, int i)
{
    for(int j = 1; j < k; j++)
    {
        if(x[j] == i || abs(x[j] - i) == abs(j - k)) return 0;
    }
    return 1;
}
void nqueen(int k, int n)
{
    for(int i = 1; i <= n; i++)
    {
        if(place(k, i))
        {
            x[k] = i;

            if(k == n)
            {
                count++;
                printf("\nSolution %d:\n", count); for(int j = 1;
                j <= n; j++)
                    printf("%d ", x[j]);
```

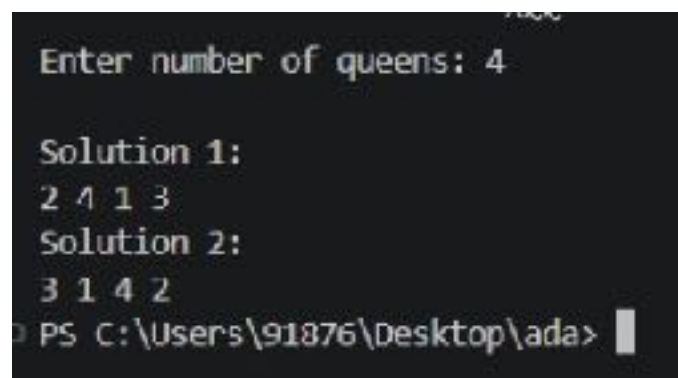
```

        }
    else
    {
        nqueen(k + 1, n);
    }
}
}

int main()
{
    int n;
    printf("Enter number of queens: ");
    scanf("%d", &n);
    nqueen(1, n); return 0;
}

```

Output>>>



```

Enter number of queens: 4

Solution 1:
2 4 1 3
Solution 2:
3 1 4 2
PS C:\Users\91876\Desktop\ada>

```